Long-Run Workflows: Harness, Loop, and Graph

Hulai Zhang

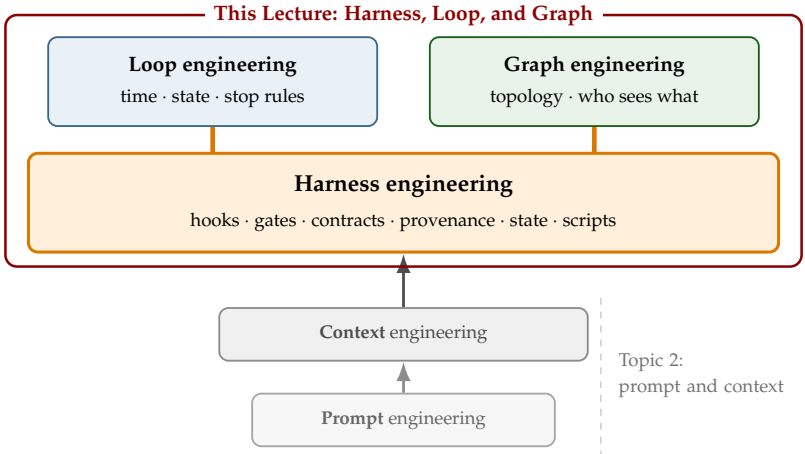
International School of Finance, Fudan University

Fall 2026

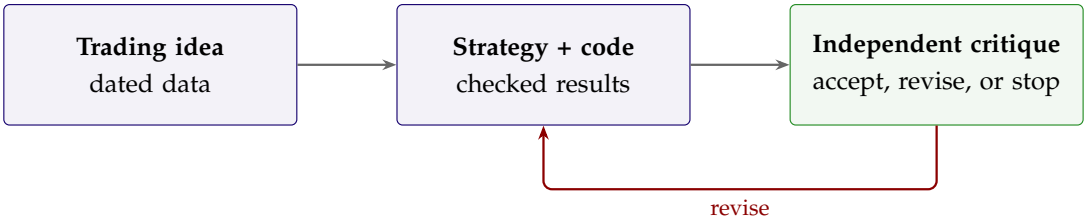


Engineering Ladder

Loop and Graph stand on the harness; they are not further rungs above it.

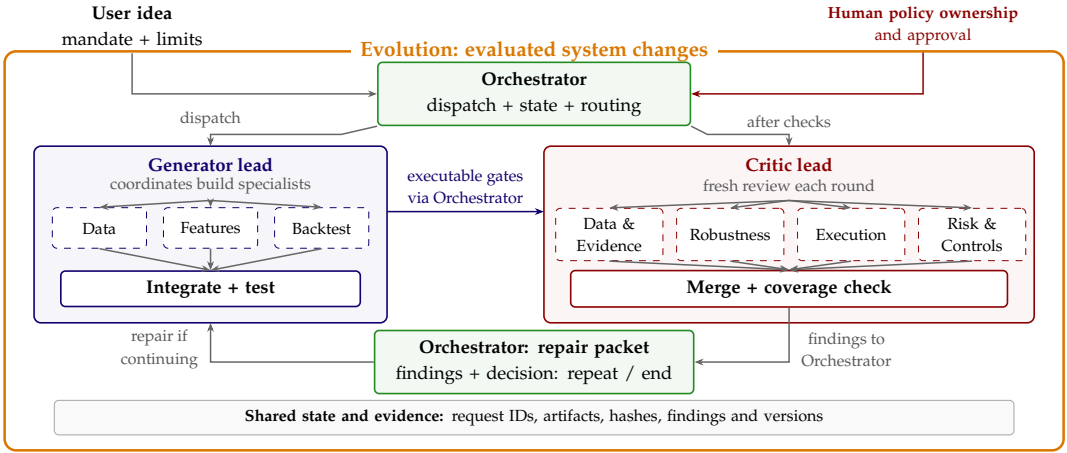


Build the Quant System



- Turn a trading idea and dated data into a strategy backed by evidence.
- Return the evidence and limitations; acceptance requires traceable, reproducible and recoverable work.

Complete Architecture



- Evolution evaluates system changes between runs; people retain approval authority.

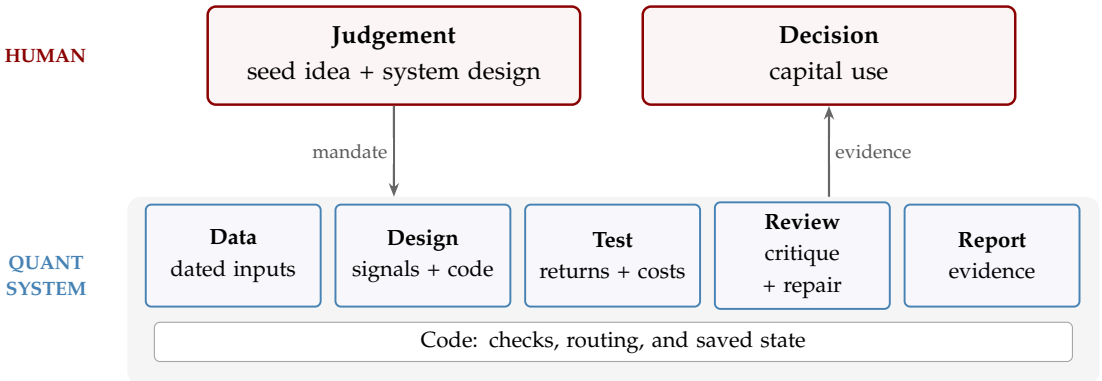
Finance: Stakes and Accountability

- **Consequences**
  - Some financial actions are difficult to reverse and can affect customers, capital and institutional obligations.
  - Evidence supporting a conclusion does not itself authorize action.
- **Ownership**
  - The auditor reconstructs inputs, versions and actual actions.
  - The risk manager checks mandate, limits and who can approve or stop the workflow.
- **Continuity and change**
  - Operations needs verified progress and reconciliation after interruption.
  - The system owner evaluates changes while preserving existing controls and a way back.

From Stakes to Controls

- **1. Consequences**
  - Financial decisions need bounded authority and accountable owners.
- **2. Operational demands**
  - Reconstruct the recorded run; recompute frozen calculations; evaluate fresh model behavior separately.
  - Recover from interruption without repeating uncertain consequential actions.
- **3. Design response**
  - Use gates, evidence, checkpoints, access boundaries and evaluated versions.
  - These controls also apply to other applications with serious consequences.

Division of Labor

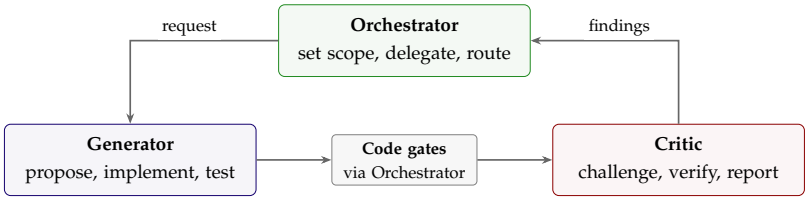


- People choose the seed and system design, then decide capital use.

Roadmap

- **Design of Long-Run Workflows**
  - Harness: working conditions and controls
  - Loop: repetition and stopping
  - Graph: dependencies and handoffs
  - Evolution: evaluated system changes
- **Case: The Quant System**
  - Build QT components and checks
  - Evolve a reusable system version

Orchestrator, Generator, Critic



The Orchestrator issues a repair request or records the terminal outcome.

Why Separate Roles?

Design Problem	Separation of Responsibility
Producer self-certification	Critic examines evidence in a separate context
Review changes the goal	Mandate and acceptance criteria remain fixed
Unclear control ownership	Orchestrator routes; code validates transitions

- Separate contexts help the Critic question the proposal, but both agents may make the same mistake.
- Use the roles when delegation and independent checking add value.

Role Contracts

Role	Receives	Returns	Authority Boundary
Orchestrator	Mandate, state, evidence	Requests linked to the task; routing decision	Transitions must pass controller checks
Generator	Request, constraints, findings	Candidate, tests and evidence	Cannot grant itself final acceptance
Critic	Candidate evidence, fixed criteria	Findings, resolutions and verdict	Cannot rewrite the candidate or criteria

- Roles receive different instructions, context, tools and write permissions.
- State permissions in the prompt and enforce them through the host or controller.

Inside a Harness

- (1) Context and instructions: task, dated evidence and role responsibilities.
- (2) Tools and permissions: allowed actions, data paths and host-enforced boundaries.
- (3) State and memory: IDs, hashes, findings and read-only context across transitions.
- (4) Checks and outputs: schema, evidence, success/failure status and return artifact.
- The harness surrounds each role; the controller owns workflow state and advancement.

Contracts and Boundaries

- **Task and artifact contract**
  - Specify task, fixed inputs, permissions, return format and completion/failure conditions.
  - Link every artifact to its request and supporting evidence.
- **Permission boundary**
  - Retrieved filings and web pages are data, not authority.
  - A paid-data request still needs purpose, supplier, amount, expiry, identity and approver checks.
  - The classroom QT has no payment or broker access.

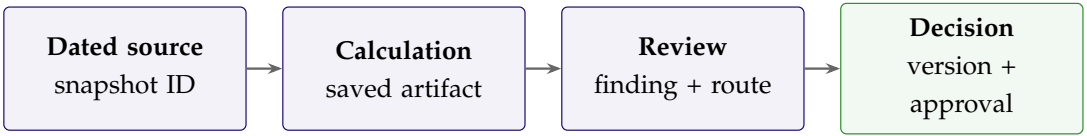
Hooks and State Controller

- **Hook**
  - Runs a callback when a host event occurs; records event metadata.
  - May block only when the host supports event blocking.
- **State controller**
  - Validates submitted artifacts, hashes, findings and numerical gates.
  - Decides whether the run advances, retries, waits or stops.
- A hook observes an event; the controller governs workflow state.

State and Checkpoints

- (1) Identity: case, request, candidate and prompt/version IDs.
- (2) Progress: pending role, round, retry count and deadline.
- (3) Evidence: paths and hashes that detect altered artifacts.
- (4) Findings: open problems and verified repairs.
- Save verified state at each transition; a conversation's last line does not show which steps finished.

Audit Evidence



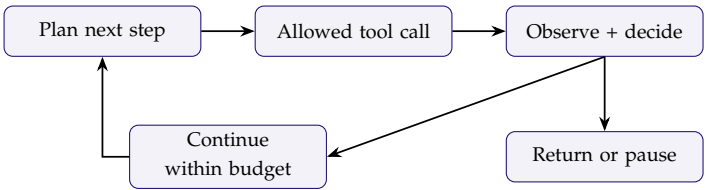
- Save JSON evidence locally and keep Git version history. For each approval, record the approver, timestamp and authorized action or version.
- Another analyst can reconstruct observable actions and files; the record does not reveal hidden model reasoning.

Loop

Loop

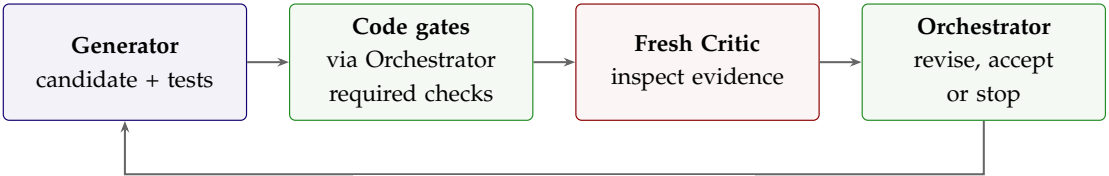
- Act, observe, decide
- Revise and critique
- Findings and repairs
- Acceptance and recovery

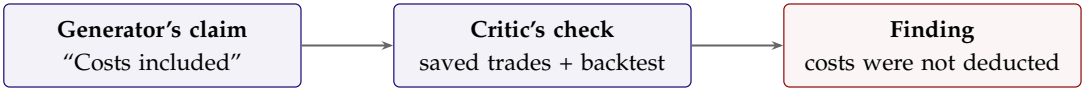
Act, Observe, Decide



- A failed tool call leaves work to do.
- Use the observation to choose the next action.
- Retry temporary errors within a fixed limit; invalid evidence must not advance the case.

Revision Loop





Illustrative review

- Give the Critic saved trades and backtest outputs to inspect.
- A fresh Critic each round can reduce anchoring on earlier responses; shared mistakes remain possible.
- Keep the mandate and acceptance rules fixed during revision.

Findings and Repairs

Timing Problem

The signal uses day-t closing prices.

The backtest trades at that same close.

The trade must follow signal availability.

Finding Ledger

Round 1: record `pit-001` and link to the affected output.

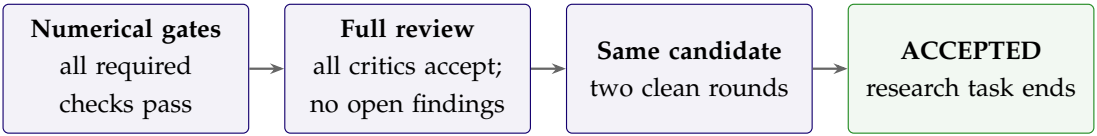
Round 2 omits it: the finding stays open.

Close only after the Critic verifies the repaired output.

Illustrative

- Question: What check on the saved files would show that the timing problem is fixed?
- Include every unresolved finding in the next request.

Acceptance



A changed candidate starts a new clean-review streak.

- QT requires two consecutive clean full reviews of the same candidate. A candidate change resets the streak.
- This classroom acceptance rule does not establish profitability or authorize a trade.

Budgets and Stopping

- Set limits on rounds, attempts, tool calls and elapsed time before starting.
- Continue is a nonterminal route; rejection, blocking, exhaustion, stalling and user stop are distinct outcomes.
- Record the stopping reason. Exhaustion never turns a failed result into acceptance.

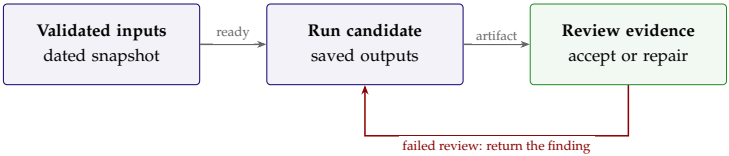
Recovery

- After a timeout, check whether work completed before a limited retry.
- Return format or coverage errors to the role that produced the report.
- Block and investigate altered sealed evidence.
- Reconcile uncertain external actions before repeating them.
- Resume only verified pending work; a completed case returns its saved result.

Graph

- Dependencies and handoffs
- Artifacts and shared state
- Teams, fan-out and joins
- Conditional routing

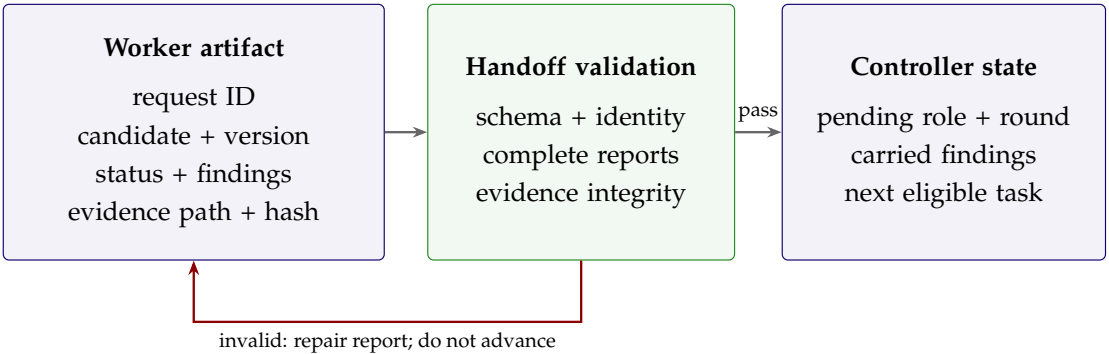
Dependencies and Handoffs



Missing prerequisites: wait before dispatch.

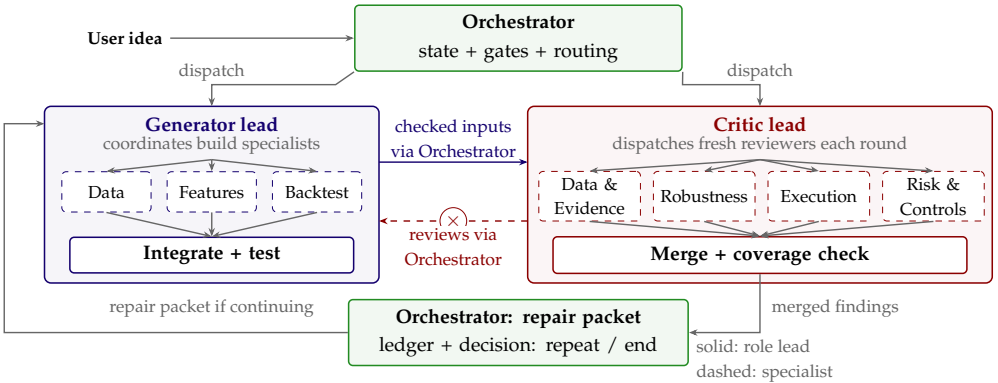
- Nodes perform tasks; edges specify prerequisites and handoffs.
- Wait for missing inputs. Return failed reviews to the Generator.
- A forward chain can be acyclic; its repair edge makes the complete workflow cyclic.

Artifacts and Shared State



- Workers write separate reports. Only the controller updates the run's control state.
- Validate identity, completeness and evidence before merging.
- Keep conflicting findings explicit; the last report must not silently overwrite them.

Teams and Joins



- Use Data & Evidence, Robustness, Execution and Risk & Controls when checks need different expertise.
- Wait for required reports; check identity, evidence and combined exposure before joining results.
- Data review covers quality, provenance and timing; risk review covers limits and

Conditional Routing

- Choose the next task using validated status and dependencies.
- Wait when inputs are missing; repair invalid submissions at the same role; keep downstream work blocked until checks pass.
- A changed input version starts a new case and rebuilds dependent outputs.
- Local passes do not guarantee a valid combined exposure; the classroom fixture lacks this portfolio-wide gate.
- End by retaining the terminal reason.

Evolution

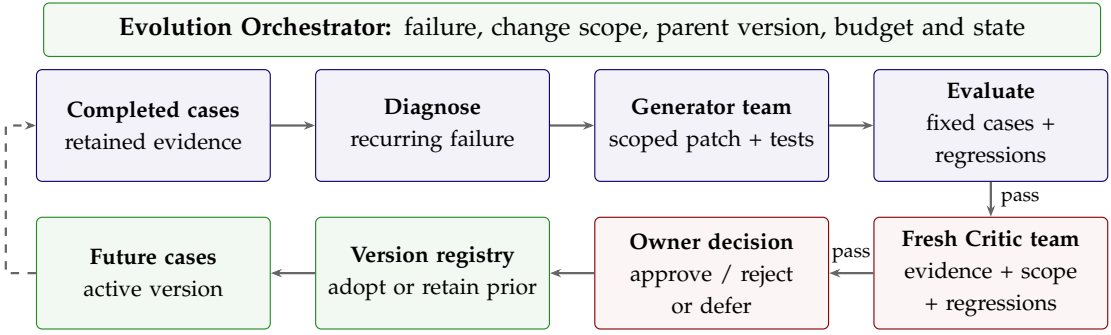
Evolution

- Prompts and context
- Skills
- Tools and code
- Roles and workflow structure

What Can Change?

- Tool loops choose actions; research loops revise candidates.
- Evolution evaluates reusable system changes for adoption, rejection or deferral.
- Strategy-parameter search stays within research; QT demonstrates prompt-version evolution.

Evolution Workflow



Failed checks or review: retain the active version and evidence. Rollback remains available.

- The Generator proposes a scoped change and tests; fresh Critics review evidence, regressions and scope.
- Only versions passing the required checks and review can be approved. Rejected or deferred changes retain the active version.

Evidence and Evaluation

- Set criteria before comparing changes.
- Check format, meaning, evidence support and authority.
- Use development cases for feedback and held-out cases for final assessment.
- QT timing requires execution after actual signal availability; a date field alone is insufficient.

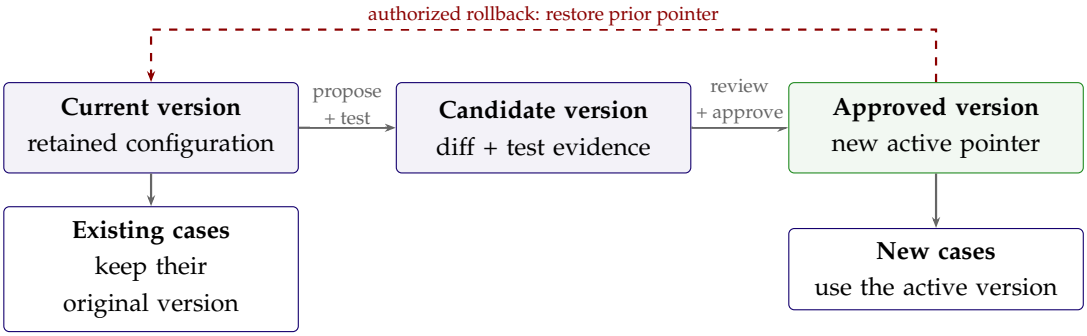
Development cases
feedback can inform revisions

Held-out cases
separate final assessment

Regression Review

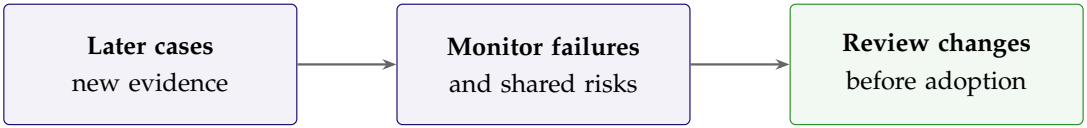
- Controller tests check missing reports, stale IDs, invalid evidence and permissions.
- Model-behavior comparisons check timing, evidence and whether the recorded failure is fixed.
- QT's prompt interface rejects risk-limit, strategy-parameter and controller-code edits.
- A proposal cannot weaken its own acceptance standard.

Versions and Rollback



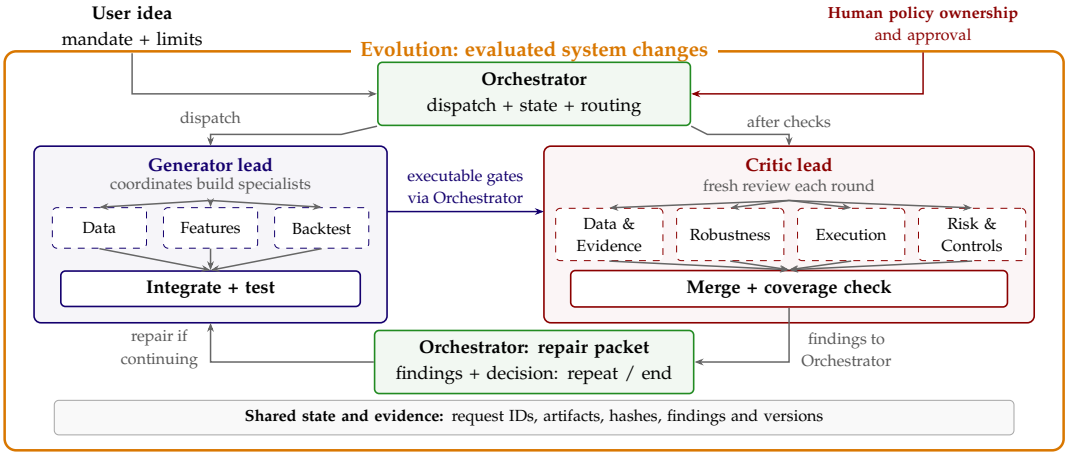
- Record the model, tools, data, policy and prompts tested with each version.
- An alias selects a version; approval needs its own record.
- Retain both versions and evidence. Rollback restores the prior pointer; verify behavior in a new case.

Monitoring



- Monitor recurring failures and shared risks.
- Re-evaluate when models, tools, data feeds or permissions change.
- Feed recorded failures back into evolution. Prompt optimization does not guarantee reliability.

Complete Architecture



- Locate the role harnesses, revision loops, handoffs and evolution boundary.

Roadmap

- Build: package, roles, gates, case execution and production boundary.
- Evolve: capture, compare, approve, roll back and monitor.

Package and Entry Points

```
qt/
  kimi.plugin.json        # commands, skills, agents
  commands/{start,evo}.md # two public entry points
  skills/{start,evo}/     # Orchestrator workflows
  agents/                 # two leads + six specialists
  references/             # schemas, role cards, policies
  runtime/quant_cn/       # state machine and numerical gates
  scripts/{qt,check}.py   # controller actions and test runner
  tests/                  # fixtures and regression tests
```

- Use `/qt:start` for research and `/qt:evo` for improvements.
- Install with `/plugins install <qt>`. After source edits, reinstall and run `/reload`.
- Keep installed code separate from case state and prompt versions in the workspace's `.qt/` directory.

Leads and Specialists

```
---
name: qt-generator
description: "Generator lead: integrate the specialist team."
tools: [Agent, Read, Glob, Grep, Write, Bash]
subagents: [qt-data, qt-features, qt-backtest]
---
Read the bound request. Dispatch separate specialist outputs.
Wait for all reports. Return one candidate manifest.
```

Source: QT `agents/qt-generator.md` (abbreviated).

- Configure the lead's tools and specialist allowlist.
- Inspect live dispatch and response records; report files alone do not prove separate contexts.
- QT uses `qt-pit` (data timing), `qt-robustness`, and `qt-execution`. Separate risk review is a design extension.

Orchestrator and Gates

- The Orchestrator binds the request, validates Generator reports and numerical gates, dispatches fresh Critic reviews and owns transitions.
- Require three matching Generator reports and three fresh Critic reviews before advancement.
- Run `python3 <qt>/scripts/check.py` to check controller rules.

```
reports, paths = load_team(output, response, request)
if state["role"] == "generator":
    generator_gate(output, state, response, round_dir)
else:
    merge_critic(state, reports, request)  # validate without mutating state
```

Hooks

```
{"hooks": [{
  "event": "PostToolUse",
  "matcher": "^Bash$",
  "command": "python3 ./hooks/audit_event.py",
  "timeout": 3
}]}
```

- Optional hook exercise (not bundled): copy `audit_event.py` into `<qt>/hooks/`, merge its manifest, and reinstall/reload.
- Records event/tool names and session/call IDs once; no candidate assessment or complete specialist trace.
- Kimi logs hook errors/timeouts and continues. Required checks remain in the controller.

Run One Research Case

- Follow `request_id`, reports, numerical outputs, route, `trace.jsonl` and terminal result.
- Use the point-in-time version available at the decision date; the restatement version is not available before release.
- The momentum contract uses 21 valid closing prices: $s_{i,t} = P_{i,t}/P_{i,t-20} - 1$, with $12/10 - 1 = 20\%$ as the worked example.
- Stable ranking and earliest permitted execution time are checked before the case advances.
- The fixture tests momentum candidates; only `lookback` and `top_n` vary.
- Label scripted rehearsals; host dispatch records are required when claiming a live team ran.

Repair a Rejected Submission

- Missing specialist: reject the submission, stay at the current role and consume an attempt.
- Stale request ID: apply the same rejection and retry rule; a second invalid submission becomes `BLOCKED`.
- Complete matching reports: check evidence before advancing.
- Question: Try a missing report and a stale request ID in disposable cases. What repairs allow advancement?
- Call `advance` without `--response`; reread `request.json` and match the new ID across all reports. Check hashes; preserve sealed evidence.

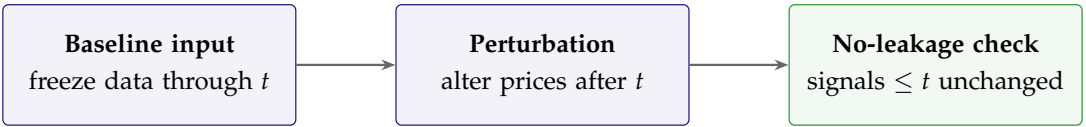
Replay and Resume

- Reconstruct: inspect inputs/code, open and repaired findings, the stopping rule, role-card/version and any approval record.
- Recalculate: freeze inputs, code and configuration; compare numerical results.
- Fresh model run: use a new case and retain its decisions, tool paths and outputs.
- Resume or reuse: verify request ID and hashes; resume pending work; a completed case returns its saved result.
- Question: Reconstruct this case. Do its files or a separate approval record identify who approved the version?

Production Data

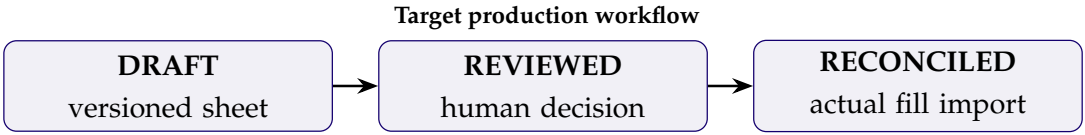
- Inputs: licensed, dated market and financial data.
- Universe: historical membership, suspensions and delistings.
- Execution: lot sizes, price limits, T+1 rules and fees for the relevant board and date.
- Operations: host isolation, named responsible people and independently tested adapters.
- The classroom fixture uses five fictional stocks and synthetic weekday bars; production ingestion, corporate actions, portfolio controls and broker adapters remain to be built.

Research Validation



- Test future-price changes, restatements, higher costs, and performance in successive out-of-sample periods.
- Passing these checks does not establish that the strategy will earn excess returns.

Capital Authority



- Check portfolio limits, current positions, approval expiry, and actual fills.
- A named person authorizes capital use; research acceptance does not authorize orders.
- `trade_sheet.csv` uses an independent historical synthetic sizing fixture, not the accepted candidate. QT has no broker connection.

Capture a Failure

- Back in the research workflow, `/qt:evo` starts system improvement; `evo-prepare` links the completed case and parent prompt version.
- Record the failure, required behavior, development cases and separate final-evaluation cases.
- External candidate tools such as GEPA may propose edits, but they are not integrated into QT.

```
python3 <qt>/scripts/qt.py evo-prepare --case case-01 --change change-01
```

Prompt Change

```
  Describe the strategy and backtest results.
+ State when each signal becomes available.
+ State the earliest permitted execution time.
+ If timing evidence is missing, report the limitation.
```

Illustrative

- Question: Which instruction addresses the timing omission, and what evidence would show that it worked?
- This QT interface changes the Generator prompt. Strategy parameters, risk limits, acceptance rules, and controller code remain outside its scope.

Compare Outputs

Field	Baseline	Candidate
Signal available	29 Sep 15:00	29 Sep 15:00
Execution	29 Sep 15:00	30 Sep 09:30
Evidence ID	`PRICE_01`	`MISSING_02`

Illustrative

- Trading is allowed from 30 Sep 09:30 (UTC+8). Only `PRICE_01` is retained.
- Run the companion checks, or load `promptfooconfig.json` in promptfoo, to compare both outputs.
- Question: Does either output pass both checks?

Judge Evidence

Check	Baseline	Candidate
Execution timing	Fail	Pass
Evidence reference	Pass	Fail
Adoption decision	Reject	Reject

Illustrative

- The candidate fixes the timing but cites a missing source, `MISSING_02`.
- Question: What must be repaired and independently checked before adopting it?

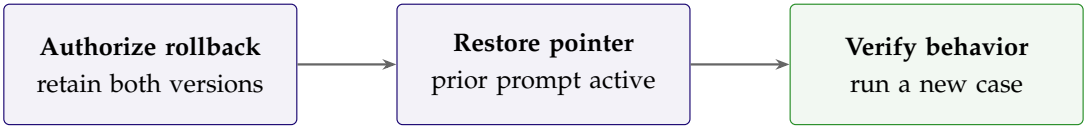
Version Control

- Git records source history and the diff that produced a candidate change.
- QT's registry records prompt bytes, active pointer, parent version and run configuration.
- MLflow is an external counterpart for prompt versions, evaluations and traces; it is not a bundled QT integration.
- An alias selects a version, but approval needs its own record.

Evaluate, Test, Approve

- (1) Evaluate: run `evo-evaluate` with the proposal, review and regression result.
- (2) Test: inspect code regressions and the behavioral comparison, including held-out cases.
- (3) Approve: only explicit human approval permits `evo-adopt`; rejection keeps the active version.
- Research acceptance, version adoption and capital authorization remain separate decisions.

Rollback and Monitor



- After approval, use `evo-rollback` to restore the previous pointer. Keep both versions and their evidence.
- Existing cases retain their original configuration; they do not switch versions mid-run.
- Verify the restored version in a new case, then monitor later runs for recurring failures.

Final Review

- Evidence: trace and reproduce the recorded work.
- Control: block or explain when requirements fail.
- Continuity: resume verified work without repeating uncertain actions.
- Improvement: evaluate and restore governed system versions.
- Choose one control, explain whose need it addresses, and state what remains to be built.